LM Studio Agent —
環境構築・型チェック・セキュリティ導入手順書

Windows 11 / Python 3.11 / GraphRAG / Vector RAG / mypy / pre-commit / Gitleaks

本書はプロジェクトZIPに含まれるREADME、docs、設定ファイルと合わせて使用してください。APIキーやモデル重み、個人データはZIPに含めません。

1. 必要なソフトウェア

- Windows 11 と Python 3.11（64-bit）。Pythonのインストール時にPATH設定を有効化。
- Git for Windows。Git BashまたはPowerShellを使用できます。
- ローカル推論を行う場合は LM Studio。クラウドAPIのみ使う場合はLM Studioは必須ではありません。
- ローカル推論では、GPU/ドライバー/LM Studioバックエンドの組合せによりAMD GPUオフロード可否が異なるため、LM Studioのロード画面で確認します。

2. 仮想環境とPythonライブラリ

```
cd C:\projects
# ZIP■■■■■■■■■■■■■■■■■■■
cd .\LMStudio-gemma-and-qwen
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
Copy-Item .env.example .env
```

requirements.txt は実行時ライブラリ（OpenAI SDK、Gradio、python-dotenv、APScheduler、pytest）を定義します。requirements-dev.txt はそれに pre-commit と mypy を追加します。依存関係を更新した場合は、同じ仮想環境で再インストールしてください。

3. .env と LM Studio

```
LM_STUDIO_BASE_URL=http://127.0.0.1:1234/v1
LM_STUDIO_MODEL=
LM_STUDIO_API_KEY=lm-studio
# ■■■■■■■■■■■■■■■LM Studio■■■■■■■■■■■■■■■ID■■■
EMBEDDING_MODEL=
# ■■■■■■■■API■■■■■■
OPENAI_API_KEY=
GEMINI_API_KEY=
TYPESAFE_API_KEY=
```

LM Studioでローカル APIサーバーを起動し、使用するチャットモデルをロードします。モデル一覧の確認例:

```
Invoke-RestMethod http://127.0.0.1:1234/v1/models
```

ベクトル検索を使うには、埋め込みAPIに対応するモデルをLM Studioにロードし、/v1/modelsに表示される実際のモデルIDをEMBEDDING_MODELへ設定します。設定しない場合でもキーワード検索とGraphRAGは利用できます。

4. アプリとテスト

```
# UI■■■■■■■■■■■■■■■■■■■■
.\start_ui.bat
.\start_worker.bat

# ■■■■■■■■■■
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m mypy --config-file mypy.ini
.\.venv\Scripts\python.exe -m compileall -q src app.py config.py worker.py evaluation tests
```

ブラウザーで http://127.0.0.1:7860 を開きます。オンラインモデルを選択すると、チャット履歴や検索結果の抜粋が選択先のAPIに送信される場合があります。明示的な同意と料金・プライバシーの確認を行ってください。

5. 型ヒントと mypy

アプリケーションのsrc/、app.py、config.py、worker.py、evaluation/の関数は引数と戻り値に型注釈を付けています。SDKやSQLiteの動的レスポンス境界にはAnyを使用する箇所があります。可能な箇所は具体的な型を優先し、Anyを新たに広げないでください。

mypy.iniではPython 3.11、check_untyped_defs、disallow_untyped_defsを有効化しています。これにより、設定対象ファイルで未注釈の関数定義を検出し、関数本体もチェックします。CIは .github/workflows/type-check.yml で同じチェックを実行します。

```
.\.venv\Scripts\python.exe -m mypy --config-file mypy.ini
```

型エラーが出た場合は、引数・戻り値・必要な変数の型を修正してから再実行します。外部SDKの型情報が不足している場合でも、根拠なく型エラーを無効化しないでください。

6. pre-commit と Gitleaks

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt

# Git■■■■■■clone■■■
.\.venv\Scripts\python.exe -m pre_commit install

# ■■■■■■■■■■■■■■■■
.\.venv\Scripts\python.exe -m pre_commit run --all-files
```

.pre-commit-config.yamlはGitleaksによるコミット対象の秘密情報スキャンとmypy型チェックを構成します。初回はフック実行環境のセットアップでネットワーク接続が必要です。各cloneでフックを登録し直してください。GitHub ActionsでもGitleaksとmypyを実行します。

7. APIキーと秘密情報の取り扱い

- 実際のAPIキーをソースコード、テスト、README、スクリーンショットに貼らない。値はローカルの.envにだけ保存する。
- .gitignoreは、過去に追跡済みになったファイルやGit履歴の秘密情報を自動削除しません。漏えい済みキーはプロバイダー側で失効・再発行してください。
- Gitleaksは誤検知もあり得ます。実際のキーを使って動作確認せず、偽のテスト文字列で確認してください。
- フックが動かない場合は、仮想環境の有効化、pre-commit installの実行、Gitがプロジェクト配下であることを確認してください。

8. 代表的なトラブル

- ModuleNotFoundError: プロジェクトルートに移動し、.venvのPythonでrequirements-dev.txtをインストールしてください。
- LM Studio接続失敗: Local Serverが起動しているか、BASE_URLとモデルIDを確認してください。
- ベクトル検索が空: EMBEDDING_MODELのID、埋め込み対応モデルのロード、Knowledge Baseのベクトル索引構築を確認してください。
- mypy失敗: エラー行の型を修正して再実行。依存SDKの型情報が原因なら、その境界を狭く型付けし、全体のチェックを無効化しないでください。
- Gitleaksでブロック: 報告されたファイルから秘密情報を取り除き、実キーが漏れた可能性があれば失効させてください。

9. 検証範囲

ZIP作成時点でPythonソースのcompileallと、ローカル環境で実行できたテスト群を確認しています。mypyとpre-commitの実行環境をこの作成環境にインストールできなかったため、mypy実行結果とGitフックの実コミット阻止は未検証です。ユーザー環境で上記コマンドを実行し、全テスト・型チェックを確認してください。